LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 07 – ABSTRACT DAN INTERFACE



Nama: CALYA ALESITA PUTRI IRAWAN

NIM: 103132400020

Kelas: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Abstract class merupakan kelas yang menggunakan kata kunci abstract dan tidak dapat dibuat menjadi objek secara langsung. Kelas ini dapat memiliki atribut, constructor, method biasa, maupun abstract methodyang belum memiliki implementasi. Abstract class digunakan sebagai kelas induk yang nantinya diwariskan kepada kelas turunan menggunakan extends, sehingga kelas turunan dapat melengkapi method yang belum memiliki implementasi.

Interface merupakan kumpulan deklarasi method yang dapat diimplementasikan oleh suatu kelas menggunakan kata kunci implements. Interface digunakan untuk menentukan kemampuan atau aturan yang harus dimiliki oleh kelas yang mengimplementasikannya. Perbedaan utama antara abstract class dan interface terletak pada penggunaannya, yaiut abstract class dapat memiliki atribut dan method dengan implementasi, sedangkan interface digunakan sebagai kontrak implementasi method. Keduanya membantu membuat program Java lebih terstrukturdan fleksibel.

B. Guided

1. Penerapan Abstract Class pada Employee

```java
package com.guided.gd1;

// Fondasi umum: belum punya peran spesifik, jadi tidak boleh
diinstalasi langsung
public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // Setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();
}
```

```java
package com.guided.gd1;

//Bentuk konkret: perannya jelas sebagai pengawal dengan skema gaji
tetap, jadi bisa diinstalasi langsung
public class SalariedEmployee extends Employee{
    private double weeklySalary;
```

```
    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }
}
```

```
package com.guided.gd1;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Budi",
5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :

```
Nama: Budi
Gaji: Rp5000000.0
```

2. Penerapan Interface Payable

```
package com.guided.gd2;

//Fondasi umum: belum punya peran spesifik, jadi tidak boleh
diinstansiasi langsung
public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();
```

```
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
package com.guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package com.guided.gd2;

// Standar sistem: siapa pun yang mau ditagih oleh kasir wajib
mengimplementasikan interface ini
public interface Payable {
    double getPaymentAmount();
}
```

```
package com.guided.gd2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
```

```
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
package com.guided.gd2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        // Dynamic dispacth: Java otomatis mengeksekusi rumus yang
tepat sesuai objek
        for (Payable item : items) {
            System.out.println("jumlah yang perlu dibayar: Rp" +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
jumlah yang perlu dibayar: Rp3000000.0
jumlah yang perlu dibayar: Rp5000000.0
```

3. Implementasi Abstract Class dan Interface pada Sistem Pembayaran

```
package com.guided.gd3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
```

```
    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

```
package com.guided.gd3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

```
package com.guided.gd3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt() {
        System.out.println("Struk Resmi - Total: Rp" +
getPaymentAmount());
    }

    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
```

```
}
```

```
package com.guided.gd3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

```
package com.guided.gd3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DOR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }

}
```

Output :

```
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.
```

C. Unguided

1. Soal Unguided 1

Buatlah program java untuk sistem kasir bioskop yang dapat memproses pembayaran tiket regular, tiket VIP, dan paket snack dalam satu transaksi. Program menerapkan abstract class, interface, enkapsulasi, serta fitur default method dan static method pada java 8.
Program terdiri dari kelas abstrak TiketBioskop beserta turunannya, yaitu TiketReguler dan TiketVIP, interface ItemTransaksi, serta kelas PaketSnack yang mengimplementasikan interface tersebut. Seluruh objek transaksi kemudian disimpan dalam satu array bertipe ItemTransaksi[] untuk menampilkan rincian pembelian dan menghitung total pembayaran secara otomatis melalui kelas main.

Jawaban :

```java
package com.unguided;

public interface ItemTransaksi {
    double getNominalBayar();

    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi()
                + " | Total: Rp" + getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

```java
package com.unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int
jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
```

```
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    public abstract String getNamaTransaksi();

    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

```
package com.unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int
jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - "
                + getJudulFilm() + " (" + getJumlahKursi()
                + " x Rp" + getHargaDasar() + ")";
    }
}
```

```java
package com.unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int
jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP)
                * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp"
                + (getHargaDasar() + biayaFasilitasVIP) + ")";
    }
}
```

```java
package com.unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }
```

```
    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " ("
                + jumlahPorsi + " x Rp" + hargaPerPorsi + ")";
    }
}
```

```
package com.unguided;

public class Main {
    public static void main(String[] args) {

        // Uji proteksi abstract class
        // TiketBioskop tiket = new TiketBioskop("Film", 40000, 2);

        ItemTransaksi.catatLogKasir(
                "Membuka transaksi baru untuk Pelanggan #042.");

        TiketReguler reguler = new TiketReguler(
                "Interstellar", 40000, 2, 3);

        TiketVIP vip = new TiketVIP(
                "Dune: Part Two", 60000, 2, 25000);

        PaketSnack snack = new PaketSnack(
                "Combo Popcorn Caramel + 2 Soda", 1, 55000);

        ItemTransaksi[] daftarTransaksi = {
            reguler, vip, snack
        };

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
```

```
        double totalTransaksi = 0;

        for (ItemTransaksi item : daftarTransaksi) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("----------------------------------------
--------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);
        System.out.println("========================================
====================");

        ItemTransaksi.catatLogKasir(
                "Transaksi #042 selesai diproses.");
    }
}
```

Output :

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan :

Pada unguided modul 7 ini, dibuat program java berupa sistem kasir terpadu bioskop yang dapat memproses pembelian tiket dan paket snack dalam satu transaksi. Program terdiri dari interface ItemTransaksi, abstract class TiketBioskop, serta kelas turunannya yaitu TiketReguler dan mengimplementasikan interface ItemTransaksi. Setiap kelas memiliki atribut dan constructor sesuai kebutuhan, serta menggunakan enkapsulasi dengan menjadikan seluruh atribut bersifat private. Interface ItemTransaksi juga dilengkapi dengan default method cetakRincianItem() untuk menampilkan rincian transaksi dan static method catatLogKasir() untuk mencatat aktivitas kasir.

Pada class Main, dibuat tiga objek transaksi yang terdiri dari tiket regular untuk film Interstellar, tiket VIP untuk film Dune: Part Two, dan paket snack Combo PopcornCaramel + 2 Soda. Ketiga objek tersebut disimpan dalam satu array bertipe ItemTransaksi[], kemudian diproses menggunakan perulangan untuk menampilkan rincian dan menghitung total pembayaran. Berdasarkan hasil program, total transaksi

yang diperoleh adalah Rp 305.000. Dengan demikian, program berhasil menjalankan proses kasir bioskop sesuai ketentuan soal.

D. Kesimpulan

Berdasarkan praktikum modul 7 tentang Abstract dan Interface ini, dapat disimpulkan bahwa abstract class dan interface merupakan konsep dalam pemrograman berorientasi objek yang digunakan untuk membuat program lebih terstruktur dan fleksibel. Pada kegiatan guided 1, dipelajari penerapan abstract class melalui kelas Employee dan kelas turunannya SalariedEmployee untuk menampilkan nama serta menghitung gaji pegawai. Pada guided 2, dipelajari penerapan interface Payable yang memungkinkan objek Invoice dan SalariedEmployee diproses menggunakan satu tipe referensi yang sama. Selanjutnya, pada guided 3, kedua konsep tersebut dikombinasikan dengan penggunaan default method dan static method untuk menampilkan struk serta mencatat aktivitas transaksi pembayaran.

Pada kegiatan unguided, konsep abstract class dan interface diterapkan dalam pembuatan system kasir bioskop yang dapat memproses pembelian tiket regular, tiket VIP, dan paket snack dalam satu transaksi. Program menggunakan kelas TiketBioskop sebagai abstract class, interface ItemTransaksi, serta beberapa kelas yang mengimplementasikan dan mewarisi konsep tersebut. Ketiga objek transaksi disimpan dalam satu array dan diproses untuk menampilkan rincian serta menghitung total pembayaran. Hasil pengujian menunjukkan total transaksi sebesar Rp 305.000. Dengan demikian, seluruh kegiatan praktikum memberikan pemahaman mengenai penerapan abstract class, interface pewarisan, dan polimorfisme dalam membangun program Java.